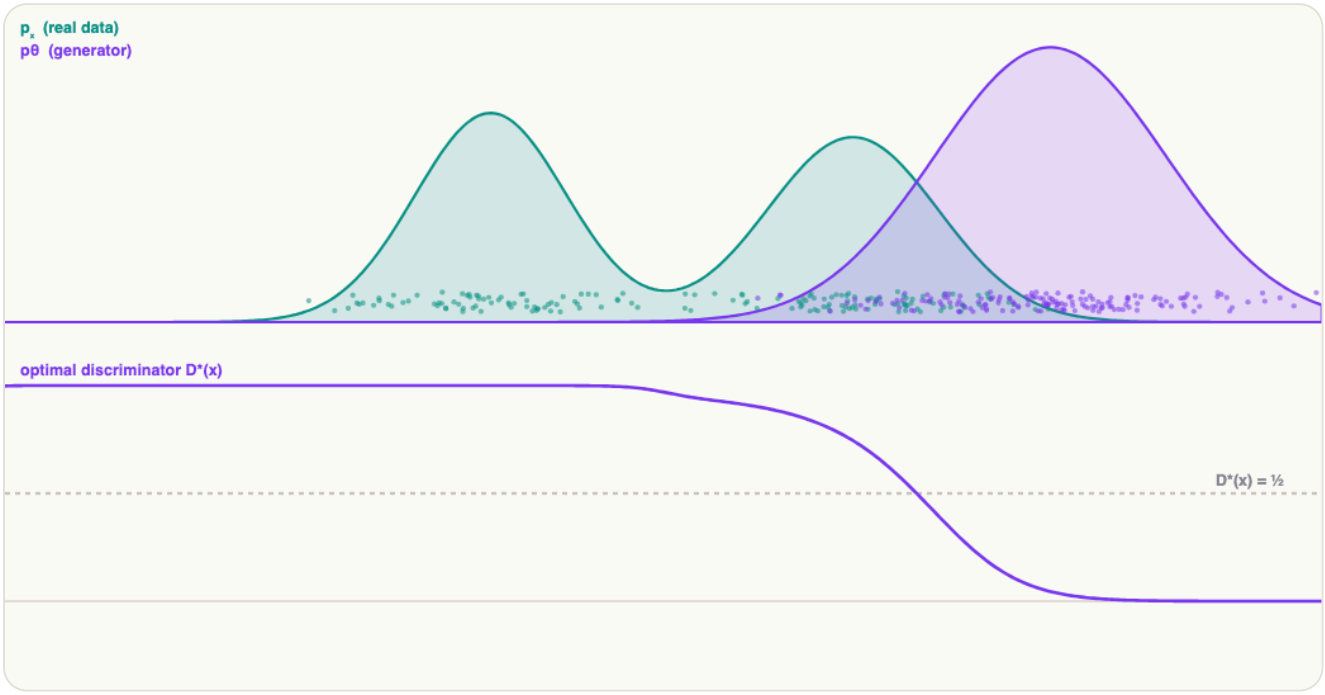
# GANs — Cheat Sheet & Visualizer

f-divergence → VDM → GAN → WGAN → BiGAN → cGAN → FID

Adversarial learning: every equation from the notes, compressed to one page you can actually revise from.

## Interactive — why the discriminator "fails" at equilibrium

Slide the generator to match the data. The *optimal* discriminator $D^*(x) = \frac{p_x(x)}{p_x(x)+p_\theta(x)}$ flattens to $\frac{1}{2}$, and $J(\theta, w)$ saturates at $-\log 4$. That flatness is the equilibrium — and also why f-divergence training stalls when the supports never overlap.




`J(θ,w) = Eₓ[log D*] + Eθ[log(1−D*)] =` **`-0.5170`** `· −log 4 = −1.3863 · gap to −log 4 =` **`0.869`**

## p7 Variational Divergence Minimization / f-Divergence

Define divergence metrics between distributions.

**f-Divergence.** Given two probability distribution functions with corresponding density functions denoted by $P_x$ and $P_\theta$, the f-divergence between them is defined as

$$D_f(P_x \| P_\theta) = \int_{\mathcal{X}} P_\theta(x)\, f\left(\frac{P_x(x)}{P_\theta(x)}\right) dx$$

$$f(u) : \mathbb{R}^+ \to \mathbb{R}, \quad \text{convex, left-semi continuous}, \quad f(1) = 0$$

$\mathcal{X}$ : space on which $P_x$ and $P_\theta$ are supported.

**Properties of f-divergence** 1. $D_f(\cdot) \geq 0$ for any choice of $f(\cdot)$ 2. $D_f(P_x \| P_\theta) = 0 \iff P_x = P_\theta$

A function $f(u)$ is convex iff $\forall u_1, u_2 \in U$ and $0 \leq \alpha_1, \alpha_2 \leq 1$:

$$\alpha_1 f(u_1) + \alpha_2 f(u_2) \;\geq\; f(\alpha_1 u_1 + \alpha_2 u_2)$$

## p8 Examples of f-divergence + Algorithm for D_f minimization

**(a)** $f(u) = u \log u$ : **KL-divergence**

$$D_f(P_x \| P_\theta) = \int_{\mathcal{X}} P_\theta(x) f\left(\frac{P_x(x)}{P_\theta(x)}\right) dx = \int_{\mathcal{X}} P_\theta(x) \left[\frac{P_x(x)}{P_\theta(x)} \log \frac{P_x(x)}{P_\theta(x)}\right] dx = \int_{\mathcal{X}} P_x(x) \log \frac{P_x(x)}{P_\theta(x)} dx = D_{KL}$$

$$D_{KL}(P_x \| P_\theta) \neq D_{KL}(P_\theta \| P_x)$$

first is *forward KL*, second is *reverse KL*.

**(b)** $f(u) = \frac{1}{2}\left(u \log u - (u+1) \log\left(\frac{u+1}{2}\right)\right)$ : **JS-divergence**

**(c)** $f(u) = \frac{1}{2}|u-1|$ : **Total variation distance**

Then: *Algorithm for $D_f$ minimization*.

## p9 Generative Models (the setup)

Data $D = \{x_1, x_2, \ldots, x_n\}$ iid $P_x$, $x_i \in \mathbb{R}^d(\mathcal{X})$. **Goal:** sample from $P_x$.

$$z \sim \mathcal{N}(0, I) \longrightarrow \boxed{g_\theta(z)} \longrightarrow \hat{x} \sim P_\theta(x) \qquad \theta^* = \arg\min_\theta D_f(P_x \| P_\theta)$$

The above setup becomes a **sampler for** $P_x$ if $P_\theta = P_x$.

**Objective:** algorithm to minimize $D_f$ between $P_x$ and $P_\theta$, *without knowing both of them, but having samples from both.* - samples from $P_x$ : dataset $D$ - samples from $P_\theta$ : outputs of $g_\theta(z)$ for different $z$

**Key Idea:** Integrals involving density functions can be approximated using samples drawn from the distribution.

## p10 Law of Large Numbers

Suppose we want to compute

$$I = \int_{\mathcal{X}} h(x)\, P_x(x)\, dx \qquad \text{where } h(x) \text{ is a function and } P_x(x) \text{ the density fn.}$$

we have samples drawn iid from $P_x$: $x_1, x_2, \ldots, x_n \sim \text{iid } P_x$

$$I = \mathbb{E}_{P_x}[h(x)]$$

**Law of Large Numbers**

$$\lim_{n\to\infty} \frac{1}{n} \sum_{i=1}^{n} h(x_i) \approx \mathbb{E}_{P_x}[h(x)], \qquad x_i \sim \text{iid } P_x$$

If the f-div. can be expressed in terms of expectations of functions w.r.t. $P_x$ and $P_\theta$, then one can compute and optimize them.

**Expressing $D_f$ in terms of Expectations over $P_x$ and $P_\theta$**

$$D_f(P_x \| P_\theta) = \int_{\mathcal{X}} P_\theta(x) f\left(\frac{P_x(x)}{P_\theta(x)}\right) dx$$

Then: *Conjugate function for a convex function*.

## p11 Conjugate function

If $f(u)$ is a convex function, then there exists a **conjugate function** $f^*(t)$ defined as follows:

$$f^*(t) = \sup_{u \in \text{dom } f} \{ ut - f(u) \}$$

**Properties of the conjugate** $f^*(t)$ 1. $f^*(t)$ is also convex 2. $[f^*(t)]^* = f(u) \;\Rightarrow\; f(u) = \sup_{t \in \text{dom } f^*} \{ tu - f^*(t) \}$

Applying it:

$$D_f(P_x \| P_\theta) = \int_{\mathcal{X}} P_\theta(x)\, \underbrace{f\left(\frac{P_x(x)}{P_\theta(x)}\right)}_{u} dx = \int_{\mathcal{X}} P_\theta(x) f(u)\, dx, \qquad u = \frac{P_x(x)}{P_\theta(x)}$$

$$f(u) = \sup_t \{tu - f^*(t)\} \qquad \Longrightarrow \qquad D_f(\cdot) = \int_{\mathcal{X}} P_\theta(x) \sup_t \{tu - f^*(t)\}\, dx$$

## p12 From sup over t to sup over a function class

$$= \int_{\mathcal{X}} P_\theta(x) \sup_t \left\{ t \left[\frac{P_x(x)}{P_\theta(x)}\right] - f^*(t) \right\} dx$$

$$= \sup_{T(x) \in \mathcal{T}} \int_{\mathcal{X}} P_\theta(x) \left\{ T(x) \frac{P_x(x)}{P_\theta(x)} - f^*(T(x)) \right\} dx$$

$\therefore$ The inner optimization problem involves $x$ and the solution for it is dependent (a fn. of $x$).

$$\mathcal{T} : \mathcal{X} \to \text{dom } f^* \quad \text{(space of functions containing solutions for the inner opt. prob.)}$$

$$T(x) \in \mathcal{T}$$

$$D_f \;\geq\; \sup_{T(x) \in \mathcal{T}} \int_{\mathcal{X}} P_\theta(x) \left\{ T(x) \frac{P_x(x)}{P_\theta(x)} - f^*(T(x)) \right\} dx$$

The space of functions $\mathcal{T}$ that we are optimizing over may **not** contain the optimal $T^*(x)$, that is the soln. for the inner opti.

## p13 The variational lower bound (boxed)

$$D_f \;\geq\; \sup_{T(x)} \left[ \int_{\mathcal{X}} P_x(x)\, T(x)\, dx - \int_{\mathcal{X}} P_\theta(x) f^*(T(x))\, dx \right]$$

$$\boxed{D_f \;\geq\; \sup_{T(x)} \left[\, \mathbb{E}_{P_x} T(x) \;-\; \mathbb{E}_{P_\theta} f^*(T(x)) \,\right]}$$

## p14 Realization of VDM

Given Data $D = \{x_1, x_2, \ldots, x_n\}$ $n$ iid $P_x$

$$z \sim \mathcal{N}(0, I) \to \boxed{g_\theta(z)} \to \hat{x} \sim P_\theta(\cdot), \qquad \theta^* = \arg\min_\theta D_f(P_x \| P_\theta)$$

$$D_f(P_x \| P_\theta) \;\geq\; \max_{T(x) \in \mathcal{T}} [\mathbb{E}_{P_x} T(x) - \mathbb{E}_{P_\theta} f^*(T(x))]$$

$$\theta^* = \arg\min_\theta D_f(\cdot) \;\approx\; \arg\min_\theta \Big[\text{lower bound on } D_f\Big] \;=\; \arg\min_\theta \left[ \max_{T(x)} \Big( \mathbb{E}_{P_x} T(x) - \mathbb{E}_{P_\theta} f^*(T(x)) \Big) \right]$$

- $\arg\min_\theta$ is w.r.t. the parameters of $g_\theta(z)$

- $\max_{T(x)}$ is w.r.t. a class of functions $T(x) \in \mathcal{T}$

Represent $\mathcal{T}$ via neural networks $T_w(x)$ where $w$ are the parameters of the neu. network. With this, the objective will become:

$$\theta^*, w^* = \arg\min_{\theta} \max_{w} \left[\mathbb{E}_{P_x} T_w(x) - \mathbb{E}_{P_\theta} f^*(T_w(x))\right]$$

## p15 Implementing VDM for Generative Modeling

Setup: - $z \sim \mathcal{N}(0, I) \rightarrow \boxed{g_\theta(z)} \rightarrow \hat{x} \sim P_\theta(x)$ — **Generator Network** - $x \rightarrow \boxed{T_w(x)} \rightarrow T_w(x)$ — **critic / Discriminator Network**

$$J(\theta, \omega) = \left[\mathbb{E}_{P_x} T_w(x) - \mathbb{E}_{P_\theta} f^*(T_w(x))\right]$$

$$\theta^*, w^* = \arg\min_{\theta} \max_{w} J(\theta, \omega) \quad \longrightarrow \quad \text{Saddle point optimization. Adversarial problem.}$$

$$T_w(x) : \mathcal{X} \rightarrow \operatorname{dom} f^*, \qquad T_w(x) = \sigma_f(V_w(x))$$

where $\sigma_f$ is a **f-divergence specific activation**, $V_w(x) : \mathcal{X} \rightarrow \mathbb{R}$, and $\sigma_f(v) : \mathbb{R} \rightarrow \operatorname{dom} f$.

> **DIAGRAM** x → [V_w(x)] → ℝ → [σ_f(·)] → dom f^* ; V_w(x)=linear layer

## p16 Generative Adversarial Networks (GANs)

$$J(\theta, \omega) = \mathbb{E}_{P_x}\left[\sigma_f(V_w(x))\right] - \mathbb{E}_{P_\theta}\left[f^*(\sigma_f(V_w(x)))\right]$$

**For GANs, the f-divergence is as follows:**

$$f(u) = u \log u - (u+1)\log(u+1) \qquad \text{(similar to JSD)}$$

$$f^*(t) = -\log(1 - \exp(t)), \qquad \operatorname{dom} f^* = \mathbb{R}^-$$

$$\sigma_f(v) = -\log\left(1 + e^{-v}\right)$$

$$J_{\text{GAN}}(\theta, w) = \mathbb{E}_{P_x}\left[\log D_w(x)\right] + \mathbb{E}_{P_\theta}\left[\log(1 - D_w(x))\right]$$

$$\text{where } D_w(x) = \frac{1}{1 + e^{-V_w(x)}} \quad : \text{ sigmoid function}$$

> **DIAGRAM** z~N(0,I) → g_θ(z) → x̂~ P_θ(x̂) ; x → V_w(x) → v → (1)/(1+e^-v) → [0,1] = D_w(x)

## p17 GAN Architecture + implementation

> **DIAGRAM** z~N(0,I) → [g_θ(z)] (generator) → x̂~ P_θ(x̂); x → [D_w(x)] (Discriminator / Binary classifier, NN) → [0,1]

Neural Networks (MLP, FNN, CNN) used for both.

$$J_{GAN}(\theta, \omega) = \mathbb{E}_{x \sim P_x} \log D_w(x) + \mathbb{E}_{\hat{x} \sim P_\theta} \log(1 - D_w(\hat{x}))$$

### Implementation of GAN in practice

Input: $D = \{x_1, x_2, x_3, \ldots, x_n\} \sim$ iid $P_x$

$$w^* = \arg\max_w \mathbb{E}_{P_x}(\log D_w(x)) + \mathbb{E}_{P_\theta}(\log(1 - D_w(\hat{x})))$$

$$\approx \arg\max_w \left[\frac{1}{B_1}\sum_{i=1}^{B_1}\log D_w(x_i) + \frac{1}{B_2}\sum_{j=1}^{B_2}\log(1 - D_w(\hat{x}_j))\right]$$

$$x_1, \ldots, x_{B_1} \sim P_x, \qquad \hat{x}_1, \ldots, \hat{x}_{B_2} \sim P_\theta, \quad \hat{x}_j = g_\theta(z_j)$$

## p18 The two update rules

$$w^{t+1} \leftarrow w^t + \alpha_1 \nabla_w J_{GAN}(\theta, w)$$

one gradient-descent step through the Discriminator; $\alpha_1$ = step-size

$$\theta^* = \arg\min_\theta J_{GAN}(\theta, w) \qquad (\theta \text{ is kept a constant})$$

$$\approx \arg\min_\theta \left[\frac{1}{B_1}\sum_{i=1}^{B_1}\log D_w(x_i) + \frac{1}{B_2}\sum_{j=1}^{B_2}\log(1 - D_w(\hat{x}_j))\right]$$

The first term is **independent of** $\theta$, hence

$$\approx \arg\min_\theta \left[\frac{1}{B_2}\sum_{j=1}^{B_2}\log(1 - D_w(g_\theta(z_j)))\right]$$

$$\theta^{t+1} \leftarrow \theta^t - \alpha_2 \nabla_\theta J_{GAN}(\theta, w)$$

one gradient-descent step through the generator; $\alpha_2$ = step size; $w$ is kept a constant

## p19 To train the Discriminator

keep $\theta$ constant, $D = \{x_1, x_2, \ldots, x_n\}$

DIAGRAM $z_1 \ldots z_B_2 \rightarrow [g_\theta(z)\ (b \times d, \theta)] \rightarrow g_\theta(z_1) \ldots g_\theta(z_B_2) \sim P_\theta(\hat{x})$ (forward prop. / backward prop.)

$$x_1, x_2 .. x_{B_1} \rightarrow \boxed{D_w(\cdot)} \rightarrow \frac{1}{B_1}\sum_{i=1}^{B_1}\log D_w(x_i)$$

$$g_\theta(z_1) \ldots g_\theta(z_{B_2}) \rightarrow \frac{1}{B_2}\sum_{j=1}^{B_2}\log(1 - D_w(g_\theta(z_j))) \rightarrow \nabla_w J_{GAN}(\theta, w)$$

$$B_1 = \{x_1, x_2, \ldots, x_{B_1}\} \subseteq D \sim P_x$$

- Sample $z_1, z_2, \ldots, z_{B_2} \sim \mathcal{N}(0, I)$ - Pass $z_1 \ldots z_{B_2}$ through $g_\theta(\cdot)$ with **fixed** $\theta$. - $w^{t+1} \leftarrow w^t + \alpha_1 \nabla_w J_{GAN}$

$$J_{GAN}(\theta, w) = \left[\frac{1}{B_1}\sum_{i=1}^{B_1}\log D_w(x_i) + \frac{1}{B_2}\sum_{j=1}^{B_2}\log(1 - D_w(g_\theta(z_j)))\right]$$

## p20 To Train the Generator Network

Sample $z_1 \ldots z_{B_2} \sim \mathcal{N}(0, I)$

DIAGRAM $z_1 \ldots z_B_2 \rightarrow [g_\theta(z)] \rightarrow g_\theta(z_1) \ldots g_\theta(z_B_2) \sim P_\theta \rightarrow (1)/(B_2)\sum_j{=}1\hat{}B_2 \log(1\text{-}D_w(g_\theta(z_j))) \rightarrow [D_w(\cdot)]$ (fixed w) $\rightarrow \nabla_\theta J_GAN(\theta, w)$

$$\text{update } \theta \text{ only, with } w \text{ a constant}$$

$$J_{GAN} = \left[\frac{1}{B_2}\sum_{j=1}^{B_2}\log(1 - D_w(g_\theta(z_j)))\right]$$

$$\theta^{t+1} \leftarrow \theta^t - \alpha_2 \nabla_\theta J_{GAN}(\theta, w)$$

Section header that follows: **Training VDM or GAN**.

## p21 Interpretation of a GAN as Classifier-Guided Generative Sampler

$$D = \{x_1, x_2, \ldots, x_n\} \sim \text{iid } P_x$$

DIAGRAM z~N(0,I)→ [g_θ(z)]→ x̂~ P_θ(x̂); (x)/(x̂) → [D_w(x)] → (1)/(0)

**Goal:** $P_\theta(\hat{x})$ to be close to $P_x(x)$.

Suppose there is a binary classifier

$$D_w(x) = \begin{cases} 1 & \text{if } x \sim P_x \\ 0 & \text{if } x \sim P_\theta \end{cases}$$

$D_w(x)$ is a binary classifier between the samples of $P_x$ and $P_\theta$.

**Question:** Can $D_w(x)$ be used for making $P_\theta$ and $P_x$ closer?

**Answer:** Tweak $\theta$ (parameters of $g_\theta$) till the classifier "fails" to distinguish between the samples of $P_x$ and $P_\theta$.

## p22 Why classifier failure is not enough

However, failure of the classifier **need not** imply $P_x = P_\theta$.

**Counter Example:**

DIAGRAM $x \in \mathbb{R}^2$, $x_i \in \{x_i^1, x_i^2\}$; two clusters $P_{\theta_3}, P_{\theta_2}$ and the data cluster $P_x$; the linear classifier $D_{w_1}$ fails on this arrangement

$$P_x = P_\theta \;\Rightarrow\; \text{classifier fails} \qquad \text{but} \qquad \text{classifier failure} \;\not\Rightarrow\; P_x = P_\theta$$

Hence, the classifier has to be **simultaneously tweaked along with the generator**.

### Formulation of classifier guided sampler

Suppose $D_w : \mathcal{X} \to [0, 1]$. Let $D_w(x)$ represent the **likelihood of the sample $x$ coming from $P_x$**.

## p23 Combined objective for the classifier

The objective is to maximize the log-likelihood of $x$ coming from $P_x$.

$$w^* = \arg\max_w \left[\mathbb{E}_{x \sim P_x} \log D_w(x)\right] \quad : \text{ maximize the likelihood of } x \sim P_x$$

Classifier should **also** maximize the likelihood of $\hat{x}$ **not** coming from $P_x$ when $\hat{x} \sim P_\theta$.

$$\mathbb{E}_{\hat{x} \sim P_\theta} \log(1 - D_w(\hat{x})) \quad : \text{ likelihood that } \hat{x} \notin P_x$$

$$w^* = \arg\max_w \mathbb{E}_{\hat{x}\sim P_\theta} \log[1 - D_w(\hat{x})] \quad : \text{ maximizing the likelihood of } \hat{x} \notin P_x$$

**The combined objective for the classifier training:**

$$w^* = \arg\max_w \left\{ \underbrace{\mathbb{E}_{x\sim P_x} (\log D_w(x))}_{} + \underbrace{\mathbb{E}_{\hat{x}\sim P_\theta} \log [1 - D_w(\hat{x})]}_{} \right\} \equiv J(\theta, w)$$

## p24 Generator objective + DCGAN

The objective for the generator $g_\theta(z)$ is such that the classifier has to "fail". **Invert** the optimization for the classifier.

$$\theta^* = \arg\min_\theta J(\theta, w)$$

$$\theta^*, w^* = \arg\min_\theta \max_w \Big[ J(\theta, w) \Big] \quad \longrightarrow \quad \text{adversarial optimization}$$

### Deep-Convolution GAN (DCGAN)

Typical in a GAN: $z \in \mathbb{R}^k$, $x \in \mathbb{R}^d$, $k \ll d$.

> DIAGRAM Multi-layer perceptron (MLP) with layer widths 16, 32, 64 → x∈ℝ ^d → reshape in the form of image

## p25 DCGAN layers + Conditional GANs

DC-GAN: **upconvolutional / Transpose Convolutional** layers are used in the generators. Data is Images: $x \in \mathbb{R}^{r\times c\times 3}$.

> DIAGRAM z∈ℝ ^16 → Transpose/upconv → tensor 3× r× c

### Conditional GANs (c-GANs)

Data $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\} \sim$ iid $P_{xy}$

Eg: $x$ : Images, \quad $y$ : class-label (textual embedding)

**Objective:** sample from the **conditional** distribution $P_{x|y}$ (instead of $P_x$)

**Solution:** Estimate $P_{x|y}$ and make $P_\theta$ approach $P_{x|y}$. Modify the generator and discriminator to operate on the samples of the **conditional** distribution.

## p26 cGAN objective + Inference with GANs / VDM

> DIAGRAM z~N(0,I) → [g_θ(z)] → x̂|y ~ P^θ_x|y; y → one-hot encoding [0,0,...,1,0,...,0] → g_θ(z)

$$(x \text{ or } \hat{x},\ y) \to \boxed{D_w(\cdot)} \to 0/1$$

$$J(\theta, \omega) = \mathbb{E}_{x,y\sim P_{xy}} \log D_w(x, y) + \mathbb{E}_{\hat{x},y\sim P^\theta_{x|y}} \log(1 - D_w(\hat{x}, y))$$

### Inference with GANs / VDM

- **Input:** A trained generator network $g_{\theta^*}$

- **Output:** A sample from $P_x$ which is **not** present in $D$.

DIAGRAM z_test~N(0,I), y → [g_θ^*(z)] → x_test corresponding to the class-label specified by y

## p27 Wasserstein's GANs: why f-divergence is unstable

f-divergence minimization leads to **unstable training**.

**Question:** What makes GAN training unstable?

**Manifold hypothesis:** Distributions of real data (such as images) lie over a lower-dim manifold in the ambient space ($\mathbb{R}^d$) — a lower dimensional subspace.

$$\text{example: a } 28 \times 28 \text{ image where each pixel } p_{ij} = 1 \text{ if toss is heads, } = 0 \text{ if toss is tails.}$$

All the images generated via the above random experiment $\in \{0,1\}^{784}$.

Recall that $P_x$ (real) and $P_\theta$ (generated) are distributions over $\mathbb{R}^d$. Since the real data lie on lower dimensional manifolds, the **supports of $P_x$ and $P_\theta$ will not be aligned** with very high probability.

## p28 Saturation, and the fix: Wasserstein's metric

It can be shown that a **perfect discriminator** ($D_w$ with 100% accuracy) can be learned when the supports of $P_x$ and $P_\theta$ are not aligned.

$$\Rightarrow \text{ The GAN training saturates}$$

$$D_f(P_x \| P_\theta) \text{ will be independent of the generator parameters } \theta.$$

**Solution:** Use a 'softer' divergence metric, that does **not** saturate when the manifolds of the supports of $P_x$ & $P_\theta$ do not align.

### Wasserstein's Metric (Optimal Transport)

Given two distributions $P_x$ and $P_{\hat{x}}$,

$$W(P_x \| P_{\hat{x}}) = \min_{d \in \Pi(x,\hat{x})} [\, \mathbb{E}_{x,\hat{x} \sim d} \, \|x - \hat{x}\|_2 \,]$$

- $d$: a **joint distribution** between $P_x$ and $P_{\hat{x}}$ - $\Pi(x, \hat{x})$: all possible joint distributions such that

$$\int_x \Pi(x,\hat{x})\, dx = P_{\hat{x}} \qquad \int_{\hat{x}} \Pi(x,\hat{x})\, d\hat{x} = P_x$$

## p29 Transport plan as a joint distribution

Eg: Suppose $P_x$ and $P_{\hat{x}}$ are two 1-d discrete pmf.

DIAGRAM bar chart $P_x$ over $x_1 x_2 x_3 \ldots$ x_K, a "redistribution" arrow to $P_{\hat{x}}$ over $\hat{x_1}\hat{x_2}\hat{x_3}\ldots\hat{x}_L$

The mass in $P_x$ can be redistributed such that $P_x$ transforms (gets converted into) $P_{\hat{x}}$. A redistribution scheme can be represented as a **joint distribution** between $P_x$ and $P_{\hat{x}}$.

DIAGRAM /table: rows $x_1, x_2, \ldots, x_K$ (row sums $P_x$); cols $\hat{x_1}, \hat{x_2}, \hat{x_3}, \ldots, \hat{x}_L$ (col sums → $P_{\hat{x}}$);

$$\text{cell example } 0.1,\ 0.2,\ 0.6,\ \ldots,\ 0.05;\ \text{the whole table } = \Pi(x,\hat{x}),\ \text{joint dist. b/w } x, \hat{x}$$

Every re-distribution scheme is a joint distribution and is called a **"transport-plan"**. Then: *quantify the effort involved in a "transport plan"*.

## p30 Work done

Suppose some mass is moved from $x$ to $\hat{x}$. - $\|x - \hat{x}\|_2$ : distance of the movement. - $\Pi(x, \hat{x})$ : amount of mass that was moved. - $\Pi(x, \hat{x}) \cdot \|x - \hat{x}\|$ : **"work-done"** in moving the mass from $x$ to $\hat{x}$.

**Avg. work done in a transport plan:**

$$\int_{x,\hat{x}} \Pi(x, \hat{x}) \, \|x - \hat{x}\| \, dx \, d\hat{x} = \mathbb{E}_{\Pi(x,\hat{x})} \|x - \hat{x}\|$$

Given that multiple transport plans (joint dist) exist between $x$ & $\hat{x}$, which of them correspond to the **least** amount of work?

Suppose $d(x, \hat{x})$ is a transport plan between $P_x$ & $P_{\hat{x}}$:

$$\int_x d(x, \hat{x}) \, dx = P_{\hat{x}}, \qquad \int_{\hat{x}} d(x, \hat{x}) \, d\hat{x} = P_x$$

## p31 The Wasserstein metric (boxed)

$$\min_{d \in \Pi(x,\hat{x})} \mathbb{E}_{d(x,\hat{x})} \|x - \hat{x}\| = W(P_x \| P_{\hat{x}})$$

$\Pi$ : family of joint distributions s.t.

$$\int_{\hat{x}} \pi(x, \hat{x}) \, d\hat{x} = P_x, \qquad \int_x \pi(x, \hat{x}) \, dx = P_{\hat{x}}$$

Transport plan leading to the least amount of work. The closer $P_x$ & $P_{\hat{x}}$ are, the lesser $W(P_x \| P_{\hat{x}})$ will be.

**Wasserstein's metric between two distributions $P_x$ & $P_{\hat{x}}$ is given by:**

$$\boxed{W_p(P_x \| P_{\hat{x}}) = \min_{d \in \Pi} \mathbb{E}_{d(x,\hat{x})} \, \|x - \hat{x}\|_p}$$

$$\int_{\hat{x}} \Pi(x, \hat{x}) \, d\hat{x} = P_x, \qquad \int_x \Pi(x, \hat{x}) \, dx = P_{\hat{x}}$$

**Fact:** Wasserstein's metric does **not saturate** unlike f-div. when the supports of $P_x$ & $P_\theta$ do not align.

## p32 WGAN objective via Kantorovich–Rubinstein duality

DIAGRAM z → [g_θ(z)] → x̂~ P_θ, θ^* = argmin_θ W(P_x P_θ)

How to minimize $W(P_x \| P_\theta)$?

**Kantrovic-Rubenstein's duality:**

$$W(P_x \| P_\theta) = \max_{\|T_w(x)\|_L \leq 1} [\, \mathbb{E}_{x \sim P_x} T_w(x) - \mathbb{E}_{\hat{x} \sim P_\theta} T_w(\hat{x}) \,]$$

$$\|T_w(x)\|_L < 1 \quad : \text{1-Lipschitz} \qquad \Rightarrow \qquad \frac{\|T_w(x_1) - T_w(x_2)\|}{\|x_1 - x_2\|} \leq 1$$

$$\theta^*, w^* = \arg\min_\theta \max_{\|T_w\|_L \leq 1} [\mathbb{E}_{x \sim P_x} T_w(x) - \mathbb{E}_{\hat{x} \sim P_\theta} T_w(\hat{x})]$$

The above objective is **very similar to GANs** and ∴ this method of minimizing the Wass. metric is called the **W-GAN**.

## p33 WGAN in practice

DIAGRAM z → [g_θ(z)] (generator) → x̂~ P_θ; (x̂ / x) → [T_w(x)] (critic) → T_w(x)

- $T_w(x)$ has to be made **1-Lipschitz**.
- Normalize the weights of $T_w$ s.t. $\|w\|_2 = 1$ after each gradient step.

**Conclusion:** Training a WGAN is more **stable** compared to that of a naive-GAN.

## p34 Inversion with GANs

DIAGRAM z~N(0,I) → [g_θ(z)] → x̂~ P_θ(x̂)

A trained GAN is a **function from** $Z \to X$. Suppose one is interested in the **inversion** of the above function, i.e. given $x_i \sim P_x$,

**Goal:** find the corresponding $z$, i.e. $z_i : g_\theta(z_i) = x_i$

**Inversion is useful for** 1. **Feature Extraction:** given a dataset, obtain GAN-inverted vectors & use them as features for data. 2. **Data manipulation / Editing:** Suppose $x_i \sim P_x$ needs to be edited.

$$\text{first get } z_i : x_i = g_{\theta^*}(z_i) \quad \text{via inversion}$$

$$z_{edit} = f_{edit}(z_i), \qquad x_{edit} = g_{\theta^*}(z_{edit})$$

## p35 Bi-directional GAN (Bi-GAN)

**Question:** How to modify a GAN such that the inversion is possible?

### Bi-directional GAN

In a naive GAN:

$$g_\theta(z) : Z \to X, \qquad D_w(x) : X \to [0,1]$$

In **Bi-GAN**, in addition to $g_\theta(z)$ & $D_w(x)$, there is another function, called the **Encoder** or the **Invertor**:

$$E_\phi : X \to Z$$

DIAGRAM z~N(0,I)→[g_θ(z)]→ x̂~ P_θ(x̂); x→[E_ϕ(x)]→ ẑ~ P_ϕ(ẑ)

$$\begin{pmatrix} g_\theta(z),\ z \\ (x,\ E_\phi(x)) \end{pmatrix} \to \boxed{D_w(\cdot)} \to [0,1]$$

In BiGAN, the $D_w(\cdot)$ is designed to classify between the data tuples of the form

$$(x,\ E_\phi(x)) \quad \text{and} \quad (g_\theta(z),\ z), \qquad x \sim P_x,\ E_\phi(x) \sim P_\phi(\hat{z}), \quad z \sim \mathcal{N}(0, I),\ g_\theta(z) \sim P_\theta$$

In a naive GAN, $D_w$ classifies between $x$ and $g_\theta(z)$.

## p36 Bi-GAN loss and its optimum

$$\hat{z} = E_\phi(x) \sim P_\phi(z), \qquad \hat{x} = g_\theta(z) \sim P_\theta(\hat{x}), \qquad z \sim P_z = \mathcal{N}(0, I)$$

$$L_{BiGAN}(\theta, w, \phi) = \min_{\theta,\phi} \max_{w} \mathbb{E}_{x\sim P_x}\left[\mathbb{E}_{\hat{z}\sim P_\phi} \log D_w\left(x, \underbrace{E_\phi(x)}_{\hat{z}}\right)\right] + \mathbb{E}_{z\sim P_z}\left[\mathbb{E}_{\hat{x}\sim P_\theta} \log\left\{1 - D_w\left(\underbrace{g_\theta(z)}_{\hat{x}}, z\right)\right\}\right]$$

Once a Bi-GAN is trained, the $g_{\theta^*}(z)$ is used for **generation** and $E_{\phi^*}(x)$ is used for **inversion**.

It can be shown that the optima of $L_{Bi\text{-}GAN}$ is achieved when

$$P_{\hat{z}x} = P_{z\hat{x}}$$

$$P_{\hat{z}x} = \int_x P_x(x) \int_{\hat{z}} P_\phi(\hat{z}|x)\, d\hat{z}\, dx$$

$$P_{z\hat{x}} = \int_z P_z(z) \int_{\hat{x}} P_\theta(\hat{x}|z)\, d\hat{x}\, dz$$

## p37 GAN Inversion via Latent Regression

DIAGRAM z~N(0,I)→[g_θ(z)]→x̂~ P_θ(x̂); x̂→[E_ϕ(x̂)]→ẑ; ‖z-ẑ‖₂², ẑ=E_ϕ(x̂)

$$\begin{pmatrix} x \\ \hat{x} \end{pmatrix} \to \boxed{D_w} \to [0, 1]$$

$$L(\theta, w, \phi) = \underbrace{\left[\mathbb{E}_{x\sim P_x} \log D_w(x) + \mathbb{E}_{\hat{x}\sim P_\theta} \log\left(1 - D_w(\hat{x})\right)\right]}_{\text{adversarial}} + \alpha\, \mathbb{E}_{\hat{x}\sim P_\theta}\left(\|z - E_\phi(\hat{x})\|_2^2\right)$$

$\alpha$ : hyper-param.

---

## p38 Adversarial Learning for handling Domain Shift

Suppose we have data from a distribution

$$D_s = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\} \sim \text{iid } P_s$$

$D_s$ : **Source** distribution.

**Task:** classification, estimate $P_s(y|x)$.

During **test time**, data comes from a dist. $D_T$ that is different from $D_s$: $D_T \neq D_s$. Classifiers/regressors trained on $D_s$ would **fail** on $D_T$.

### Unsupervised Domain Adaptation (UDA)

$$D_s = \{(x_i, y_i)\}_{i=1}^n \sim \text{iid } P_s, \qquad D_T = \{\hat{x}_j\}_{j=1}^m \sim \text{iid } P_T, \qquad P_s \neq P_T \ \ (\text{Domain shift})$$

**Objective:** Given $D_s$ & $D_T$, learn features / classifier such that it performs well on both $P_s$ & $P_T$.

### Domain Adversarial Networks

$$\Phi : \mathcal{X} \to F, \qquad \Phi(x) = f_s, \qquad \Phi(\hat{x}) = f_t$$

DIAGRAM x & x̂ → [Φ(x)] → f_s & f_t; f_s → [h_ψ(·)] → y_s~ P_s(y|x) (classifier for the source distribution);

$$f_s \ \&\ f_t \to \boxed{D_w(\cdot)} \to [0, 1] \quad (\text{the GAN that would match } P_{f_s} = P_{f_t})$$

$$f_s \sim P_{f_s}, \qquad f_t \sim P_{f_t}$$

$$\Phi^*, w^* = \arg\min_{\Phi}\max_{w}\ \mathbb{E}_{P_{f_s}} \log D_w(f_s) + \mathbb{E}_{P_{f_t}} \log\left(1 - D_w(f_t)\right)$$

## p39 UDA objectives and inference

$$\Phi^*, \psi^* = \arg\min_{\psi,\Phi}\ \mathrm{BCE}\left(y,\ h_\psi(f_s)\right)$$

$\Phi$ has **two objectives**: (a) adversarial objective, (b) classification objective.

During **Inference**, the same classifier $h_{\psi^*}(\cdot)$ can be used on both the source & target dist. as long as the features are used in the input.

$$\text{Given } \hat{x}_{test} \sim P_T, \qquad f_t^{test} = \Phi^*(\hat{x}_{test}), \qquad \hat{y}_{test} = h_{\psi^*}\left(f_t^{test}\right)$$

Since $\Phi$ would ensure that $P_{f_s} = P_{f_t}$, $h_{\psi^*}$ will function the same way irrespective of the domain.

---

## p40 Evaluation of Generative Models

**Question:** How to validate a trained gen. model?

$$D_{true} = \{x_1, x_2, \ldots, x_n\} \sim \text{iid } P_x \quad (\text{real-data / Inp data})$$

$$D_{gen} = \{\hat{x}_1, \hat{x}_2, \ldots, \hat{x}_n\} \sim \text{iid } P_{\theta^*} \quad (\text{generated data})$$

**Goal:** Compare $D_{true}$ & $D_{gen}$.

A popular metric to compare $D_{true}$ & $D_{gen}$: **Frechet Inception Distance (FID)**

**FID :** Wasserstein's metric between $D_{true}$ & $D_{gen}$

1. Take a **pretrained Inception network** (Deep CNN) trained on Imagenet (Image data).
2. pass $D_{true}$ & $D_{gen}$ through the above Inception network.

   > DIAGRAM (x~ P_x / x̂~ P_θ^*) → [I_ψ^*(x)] → ⋯ l^th layer; l: hyper-param

   Extract the features for $D_{true}$ & $D_{gen}$

## p41 FID computation

From an $l^{th}$ layer of $I_{\psi^*}(x)$:

$$\hat{D}_{true} = \{z^1_{true}, z^2_{true}, \ldots, z^n_{true}\}, \qquad \hat{D}_{gen} = \{z^1_{gen}, z^2_{gen}, \ldots, z^n_{gen}\}$$

$$z^i_{true} = \text{output of the } l^{th} \text{ layer of } I_{\psi^*}(\cdot), \qquad z^i_{gen} = \text{output of the } l^{th} \text{ layer of } I_{\psi^*}(\cdot)$$

3. Compute mean & co-var for $\hat{D}_{gen}, \hat{D}_{true}$:

   $$\mu_{true}(z_{true}) = \frac{1}{n}\sum_{i=1}^{n} z^i_{true}, \qquad \Sigma_{true} = \frac{1}{n}\sum_{i=1}^{n} z^i_{true}\left(z^i_{true}\right)^T$$

   Similarly compute $\mu_{gen}, \Sigma_{gen}$.

4. **Assumption:**

   $$z_{true} \sim \mathcal{N}\left(\mu_{true}, \Sigma_{true}\right), \qquad z_{gen} \sim \mathcal{N}\left(\mu_{gen}, \Sigma_{gen}\right)$$

5.

$$FID = W(\mathcal{N}(\mu_{true}, \Sigma_{true}) \parallel \mathcal{N}(\mu_{gen}, \Sigma_{gen}))$$

$$= \underbrace{\|\mu_{true} - \mu_{gen}\|_2^2}_{\text{scalar}} + \underbrace{\text{tr}\left(\Sigma_{true} + \Sigma_{gen} - 2\left(\Sigma_{true}\Sigma_{gen}\right)^{1/2}\right)}_{\text{scale}}$$

## p42 Interpreting FID

The generative Model is **better if FID is lower**.

$$\text{lower FID} \Rightarrow \text{lower W. distance between } P_x \text{ and } P_{\theta^*}$$

**— End of Adversarial learning —**

---

Transcribed page-by-page from *GenAI-IITM.pdf* (handwritten notes, pages 7–42). Notation preserved as written. Built for revision — verify against the source for coursework.